

Evidencias de ejecución

Gestión de estudiantes con Node.js, JavaScript y Express

Estudiante: Miguel Ángel Jaramillo Mesa

Asignatura: Programación Avanzada

Docente:

Fecha:

Repositorio: https://github.com/Majaramill/ACA-_FINAL_PROGRAMACION_AVANZADA_54468

Contenido

1. Ejecución general del programa en consola
2. Pruebas individuales por requerimiento
3. Pruebas de la API con Express
4. Conclusión

1. Ejecución general del programa en consola

Ejecución completa de los trece requerimientos mediante el archivo `console.js`. Se muestra el listado de los diez estudiantes registrados.

node console.js — listado de todos los estudiantes

```
Problems Output Terminal Ports Debug Console
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node console.js
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node console.js

-----
1. LISTADO DE TODOS LOS ESTUDIANTES
-----
ID: 1 | Andrés | Edad: 21 | Ingeniería de Sistemas | Sem: 5 | Promedio: 4.2
ID: 2 | Laura | Edad: 20 | Administración | Sem: 4 | Promedio: 3.8
ID: 3 | Carlos | Edad: 24 | Ingeniería de Sistemas | Sem: 7 | Promedio: 2.9
ID: 4 | Mariana | Edad: 22 | Contaduría | Sem: 6 | Promedio: 4.5
ID: 5 | Juan | Edad: 19 | Ingeniería de Sistemas | Sem: 3 | Promedio: 3.2
ID: 6 | Sofía | Edad: 23 | Administración | Sem: 8 | Promedio: 4.7
ID: 7 | Daniel | Edad: 25 | Contaduría | Sem: 9 | Promedio: 2.7
ID: 8 | Valentina | Edad: 21 | Ingeniería de Sistemas | Sem: 5 | Promedio: 3.9
ID: 9 | Sebastián | Edad: 22 | Administración | Sem: 6 | Promedio: 3.4
ID: 10 | Camila | Edad: 20 | Contaduría | Sem: 4 | Promedio: 4.1
Total: 10 estudiante(s)
```

2. Pruebas individuales por requerimiento

Cada función se ejecutó de forma aislada con el archivo `pruebas.js`, que permite correr un solo requerimiento y pasarle valores distintos desde la terminal. Se probaron tanto los casos con resultados como los casos sin coincidencias.

Punto 1 — Listar todos los estudiantes

Devuelve una copia del array. La última línea comprueba que no es el array original, con lo que se cumple la regla de no modificar los datos de partida.

node pruebas.js 1 — primer y último elemento, y verificación de la copia

```
Problems  Output  Terminal  Ports  Debug Console

PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 1

Primer elemento:
{
  id: 1,
  nombre: 'Andrés',
  edad: 21,
  carrera: 'Ingeniería de Sistemas',
  semestre: 5,
  promedio: 4.2
}

Ultimo elemento:
{
  id: 10,
  nombre: 'Camila',
  edad: 20,
  carrera: 'Contaduría',
  semestre: 4,
  promedio: 4.1
}

¿Es el mismo array en memoria? false
(Es Falso porque debe ser la copia del array y no el original)
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

Punto 2 — Buscar un estudiante por ID

Con un ID existente devuelve el objeto completo; con uno inexistente devuelve null, que es lo que permite a la consola mostrar el mensaje de no encontrado y a Express responder con un código 404.

node pruebas.js 2 — ID 4, devuelve a Mariana

```
node pruebas.js 2

=====
PUNTO 2 - buscarPorId(estudiantes, 4)
=====

Devolvio el objeto:
{
  id: 4,
  nombre: 'Mariana',
  edad: 22,
  carrera: 'Contaduría',
  semestre: 6,
  promedio: 4.5
}

Acceso a sus propiedades:
.nombre -> Mariana
.promedio -> 4.5
```

node pruebas.js 2 5 — ID 5, devuelve a Juan

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 2 5
```

```
=====
PUNTO 2 – buscarPorId(estudiantes, 5)
=====
```

Devolvio el objeto:

```
{
  id: 5,
  nombre: 'Juan',
  edad: 19,
  carrera: 'Ingeniería de Sistemas',
  semestre: 3,
  promedio: 3.2
}
```

node pruebas.js 2 100 — ID inexistente, devuelve null

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 2 100
```

```
=====
PUNTO 2 – buscarPorId(estudiantes, 100)
=====
```

Devolvio null -> no existe un estudiante con ese ID.
Asi es como la consola sabe imprimir 'no encontrado'
y como Express sabe responder con un error 404.

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

Punto 3 — Buscar estudiantes por carrera

La búsqueda ignora tildes y mayúsculas gracias a la función `normalizarTexto`. Cuando no hay coincidencias devuelve un array vacío, no un error.

node pruebas.js 3 — Ingeniería de Sistemas, 4 estudiantes

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 3
```

```
=====
PUNTO 3 – buscarPorCarrera(estudiantes, "Ingeniería de Sistemas")
=====
```

Cantidad encontrada:

4

Andrés – Ingeniería de Sistemas
Carlos – Ingeniería de Sistemas
Juan – Ingeniería de Sistemas
Valentina – Ingeniería de Sistemas

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

node pruebas.js 3 Administración — Administración, 3 estudiantes

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 3 Administración
```

```
=====
PUNTO 3 – buscarPorCarrera(estudiantes, "Administración")
=====
```

Cantidad encontrada:

3

Laura – Administración
Sofía – Administración
Sebastián – Administración

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

node pruebas.js 3 Contaduria — escrito sin tilde, encuentra los 3 de Contaduría

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 3 Contaduria

=====
PUNTO 3 – buscarPorCarrera(estudiantes, "Contaduria")
=====

Cantidad encontrada:
3
  Mariana – Contaduría
  Daniel – Contaduría
  Camila – Contaduría
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

node pruebas.js 3 Soldadura — carrera inexistente, array vacío

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 3 Soldadura

=====
PUNTO 3 – buscarPorCarrera(estudiantes, "Soldadura")
=====

Cantidad encontrada:
0
(array vacío: ninguna coincidencia)
```

Punto 4 — Estudiantes aprobados

Filtra los que tienen promedio mayor o igual a 3.0. La segunda parte de la captura demuestra el parámetro opcional: con una nota mínima de 4.0 el resultado se reduce a cuatro estudiantes sin modificar el código.

node pruebas.js 4 — 8 aprobados con la nota mínima de 3.0

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 4

=====
PUNTO 4 – obtenerAprobados(estudiantes)
=====

Aprobados (promedio >= 3): 8

  Andrés – 4.2
  Laura – 3.8
  Mariana – 4.5
  Juan – 3.2
  Sofía – 4.7
  Valentina – 3.9
  Sebastián – 3.4
  Camila – 4.1

Con nota mínima 4.0 serían solo 4:
  Andrés – 4.2
  Mariana – 4.5
  Sofía – 4.7
  Camila – 4.1
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

Punto 5 — Estudiantes reprobados

Filtra los que están por debajo de 3.0. La comprobación final valida que aprobados más reprobados suman el total de estudiantes.

node pruebas.js 5 — 2 reprobados: Carlos y Daniel

```

● PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 5

=====
PUNTO 5 – obtenerReprobados(estudiantes)
=====

Reprobados (promedio < 3): 2

    Carlos – 2.9
    Daniel – 2.7

Comprobacion: 8 aprobados + 2 reprobados = 10 total
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>

```

Punto 6 — Promedio general

Se muestra la suma, la división sin redondear y el resultado final. El valor intermedio 3.7399999999999998 evidencia por qué se aplica un redondeo, y la última línea confirma que el dato devuelto es numérico y no texto.

node pruebas.js 6 — promedio general de 3.74

```

● PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 6

=====
PUNTO 6 – calcularPromedioGeneral(estudiantes)
=====

Suma de los promedios: 37.4
Dividido entre 10 = 3.7399999999999998
Despues de redondear -> 3.74

Tipo de dato devuelto: number (debe ser number, no string)
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>

```

Punto 7 — Mejor estudiante

Recorre el array comparando contra el mejor promedio encontrado hasta el momento.

node pruebas.js 7 — Sofía con 4.7

```
● PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 7
```

```
=====
PUNTO 7 – mejorEstudiante(estudiantes)
=====
```

```
Devolvio:
```

```
{
  id: 6,
  nombre: 'Sofía',
  edad: 23,
  carrera: 'Administración',
  semestre: 8,
  promedio: 4.7
}
```

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

Punto 8 — Estudiante con menor promedio

Misma lógica del punto anterior con la comparación invertida.

node pruebas.js 8 — Daniel con 2.7

```
● PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 8
```

```
=====
PUNTO 8 – peorEstudiante(estudiantes)
=====
```

```
Devolvio:
```

```
{
  id: 7,
  nombre: 'Daniel',
  edad: 25,
  carrera: 'Contaduría',
  semestre: 9,
  promedio: 2.7
}
```

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

Punto 9 — Cantidad de estudiantes por carrera

Devuelve un objeto donde cada llave es una carrera. No requiere conocer de antemano cuáles carreras existen: las llaves se crean durante el recorrido. La suma de todas coincide con el total de estudiantes.

node pruebas.js 9 — 4 de Ingeniería, 3 de Administración y 3 de Contaduría


```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 9
```

```
=====
PUNTO 9 – contarPorCarrera(estudiantes)
=====
```

Objeto devuelto:

```
{ 'Ingeniería de Sistemas': 4, 'Administración': 3, 'Contaduría': 3 }
```

Recorrido con for...in:

Ingeniería de Sistemas: 4

Administración: 3

Contaduría: 3

(suma de todos: 10, debe dar 10)

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

Punto 10 — Buscar estudiantes por semestre

Filtra por coincidencia exacta del número de semestre.

node pruebas.js 10 — semestre 5: Andrés y Valentina

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 10
```

```
=====
PUNTO 10 – buscarPorSemestre(estudiantes, 5)
=====
```

Cantidad encontrada:

2

Andrés – semestre 5

Valentina – semestre 5

```
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> |
```

node pruebas.js 10 3 — semestre 3: solo Juan

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 10 3
```

```
=====
PUNTO 10 – buscarPorSemestre(estudiantes, 3)
=====
```

Cantidad encontrada:

1

Juan – semestre 3

node pruebas.js 10 100 — semestre inexistente, array vacío

```

PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 10 100

=====
PUNTO 10 — buscarPorSemestre(estudiantes, 100)
=====

Cantidad encontrada:
0
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>

```

Punto 11 — Estudiantes mayores de cierta edad

La condición usa el operador mayor estricto, tal como pide el enunciado: los estudiantes que tengan exactamente la edad indicada quedan por fuera del resultado.

node pruebas.js 11 — valor por defecto

```

PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 11

Estudiantes con edad MAYOR a 18: 10

Andrés — 21 años
Laura — 20 años
Carlos — 24 años
Mariana — 22 años
Juan — 19 años
Sofía — 23 años
Daniel — 25 años
Valentina — 21 años
Sebastián — 22 años
Camila — 20 años

Ojo: los de exactamente 18 años quedan POR FUERA,
porque la condición usa > y no >= ya que deben ser mayor a la edad indicada.
❖ PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>

```

node pruebas.js 11 19 — mayores de 19 años

```

PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 11 19

=====
PUNTO 11 — mayoresDeEdad(estudiantes, 19)
=====

Estudiantes con edad MAYOR a 19: 10

Andrés — 21 años
Laura — 20 años
Carlos — 24 años
Mariana — 22 años
Juan — 19 años
Sofía — 23 años
Daniel — 25 años
Valentina — 21 años
Sebastián — 22 años
Camila — 20 años

```

node pruebas.js 11 26 — ningún estudiante supera esa edad, array vacío

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 11 26
```

```
=====
PUNTO 11 – mayoresDeEdad(estudiantes, 26)
=====
```

```
Estudiantes con edad MAYOR a 26: 0
```

```
Ojo: los de exactamente 26 años quedan POR FUERA,
porque la condicion usa > y no >= ya que deben ser mayor a la edad indicada.
```

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

Punto 12 — Reporte general

Esta función no realiza cálculos nuevos: reutiliza las funciones anteriores y agrupa sus resultados en un solo objeto. La segunda captura muestra el acceso individual a cada dato del reporte.

node pruebas.js 12 — objeto completo devuelto por generarReporte

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes> node pruebas.js 12
```

```
=====
PUNTO 12 – generarReporte(estudiantes)
=====
```

```
Objeto devuelto:
```

```
{
  totalEstudiantes: 10,
  aprobados: 8,
  reprobados: 2,
  promedioGeneral: 3.74,
  mejorEstudiante: {
    id: 6,
    nombre: 'Sofía',
    edad: 23,
    carrera: 'Administración',
    semestre: 8,
    promedio: 4.7
  },
  peorEstudiante: {
    id: 7,
    nombre: 'Daniel',
    edad: 25,
    carrera: 'Contaduría',
    semestre: 9,
    promedio: 2.7
  },
  estudiantesPorCarrera: { 'Ingeniería de Sistemas': 4, 'Administración': 3, 'Contaduría': 3 }
}
```

node pruebas.js 12 — acceso a cada propiedad por separado

```
Acceso a cada dato por separado:
```

```
.totalEstudiantes -> 10
```

```
.promedioGeneral -> 3.74
```

```
.mejorEstudiante.nombre -> Sofía
```

```
PS C:\Users\Miguel Jaramillo\Desktop\gestion-estudiantes>
```

3. Pruebas de la API con Express

Las mismas funciones desarrolladas para la consola se expusieron como una API REST mediante Express, sin duplicar ninguna línea de lógica: cada endpoint llama a la función correspondiente y convierte su resultado en JSON. El servidor se levanta con el comando `npm start` y las rutas se consultaron desde el navegador.

Página de inicio del servidor

La ruta raíz devuelve una página HTML con los trece endpoints disponibles como enlaces, lo que permite probarlos sin escribir las direcciones a mano.

`http://localhost:3000/` — índice de endpoints

API Gestión de Estudiantes

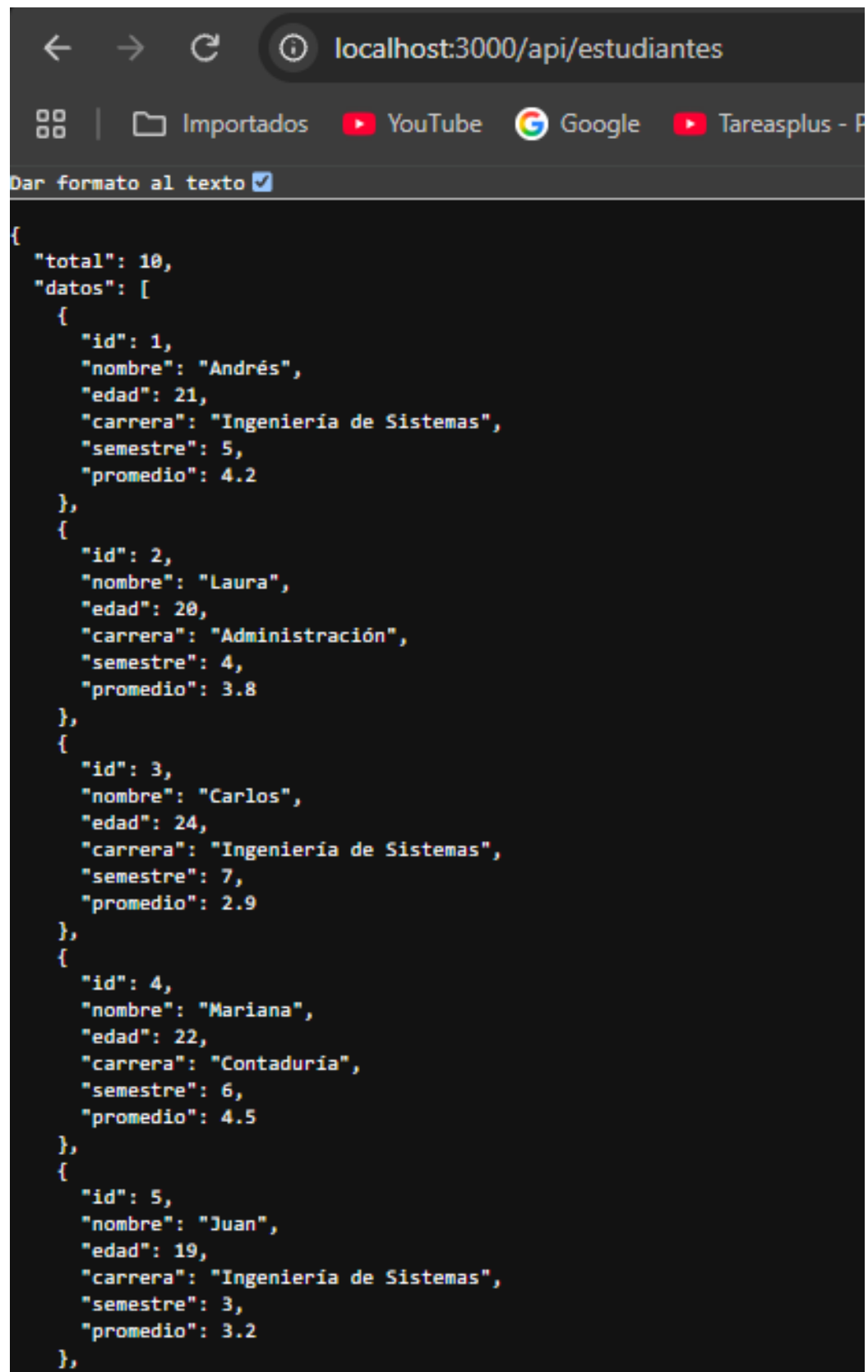
Node.js + Express. Haga clic en cualquier ruta para probarla.

GET /api/estudiantes	1. Listar todos los estudiantes
GET /api/estudiantes/:id	2. Buscar estudiante por ID
GET /api/estudiantes/carrera/:carrera	3. Buscar por carrera
GET /api/estudiantes/aprobados	4. Aprobados (promedio ≥ 3.0)
GET /api/estudiantes/reprobados	5. Reprobados (promedio < 3.0)
GET /api/estadisticas/promedio	6. Promedio general
GET /api/estadisticas/mejor	7. Mejor estudiante
GET /api/estadisticas/peor	8. Estudiante con menor promedio
GET /api/estadisticas/carreras	9. Conteo por carrera
GET /api/estudiantes/semestre/:semestre	10. Buscar por semestre
GET /api/estudiantes/mayores/:edad	11. Mayores de cierta edad
GET /api/reporte	12. Reporte académico general
GET /api/ranking	Reto: ranking por promedio

Punto 1 — Listar todos los estudiantes

Devuelve el total y el arreglo completo en formato JSON.

`/api/estudiantes` — los 10 estudiantes registrados



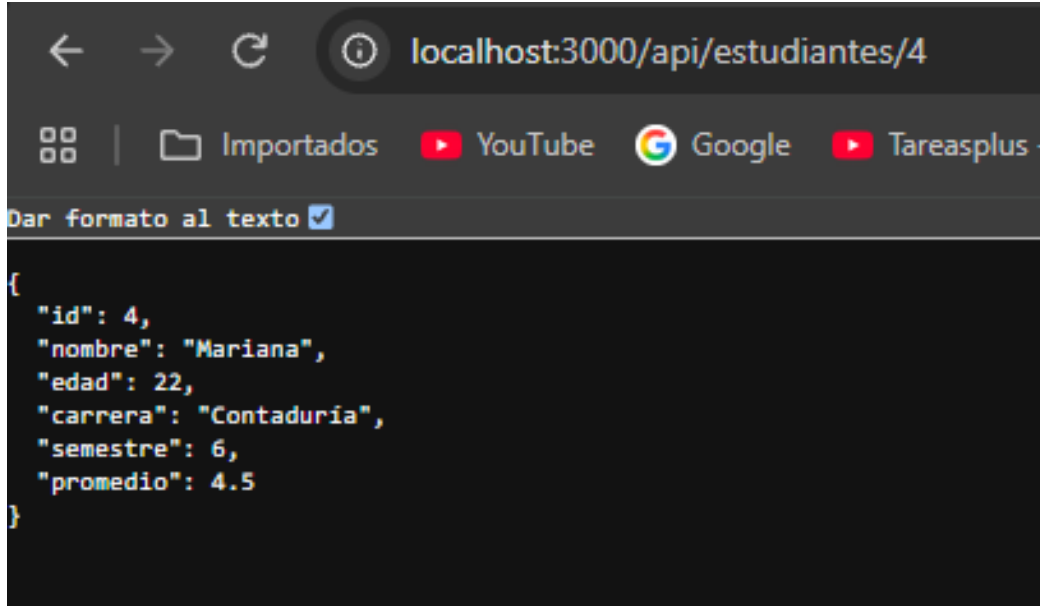
A screenshot of a web browser window displaying a REST client interface. The address bar shows the URL `localhost:3000/api/estudiantes`. Below the address bar, there are tabs for "Importados", "YouTube", "Google", and "Tareasplus - P". A checkbox labeled "Dar formato al texto" is checked. The main area of the browser displays a JSON response from the API:

```
{
  "total": 10,
  "datos": [
    {
      "id": 1,
      "nombre": "Andrés",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 5,
      "promedio": 4.2
    },
    {
      "id": 2,
      "nombre": "Laura",
      "edad": 20,
      "carrera": "Administración",
      "semestre": 4,
      "promedio": 3.8
    },
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    },
    {
      "id": 4,
      "nombre": "Mariana",
      "edad": 22,
      "carrera": "Contaduría",
      "semestre": 6,
      "promedio": 4.5
    },
    {
      "id": 5,
      "nombre": "Juan",
      "edad": 19,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 3,
      "promedio": 3.2
    },
    {
      "id": 6,
      "nombre": "Diego",
      "edad": 23,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 6,
      "promedio": 3.5
    },
    {
      "id": 7,
      "nombre": "Ana",
      "edad": 25,
      "carrera": "Administración",
      "semestre": 8,
      "promedio": 2.5
    },
    {
      "id": 8,
      "nombre": "Pedro",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 4,
      "promedio": 3.9
    },
    {
      "id": 9,
      "nombre": "Sofía",
      "edad": 20,
      "carrera": "Contaduría",
      "semestre": 5,
      "promedio": 4.1
    },
    {
      "id": 10,
      "nombre": "Mateo",
      "edad": 22,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.8
    }
  ]
}
```

Punto 2 — Buscar un estudiante por ID

El valor de la URL es un parámetro variable, por lo que una sola ruta atiende cualquier identificador. Cuando el estudiante no existe, la API responde con el código HTTP 404 y un mensaje explicativo.

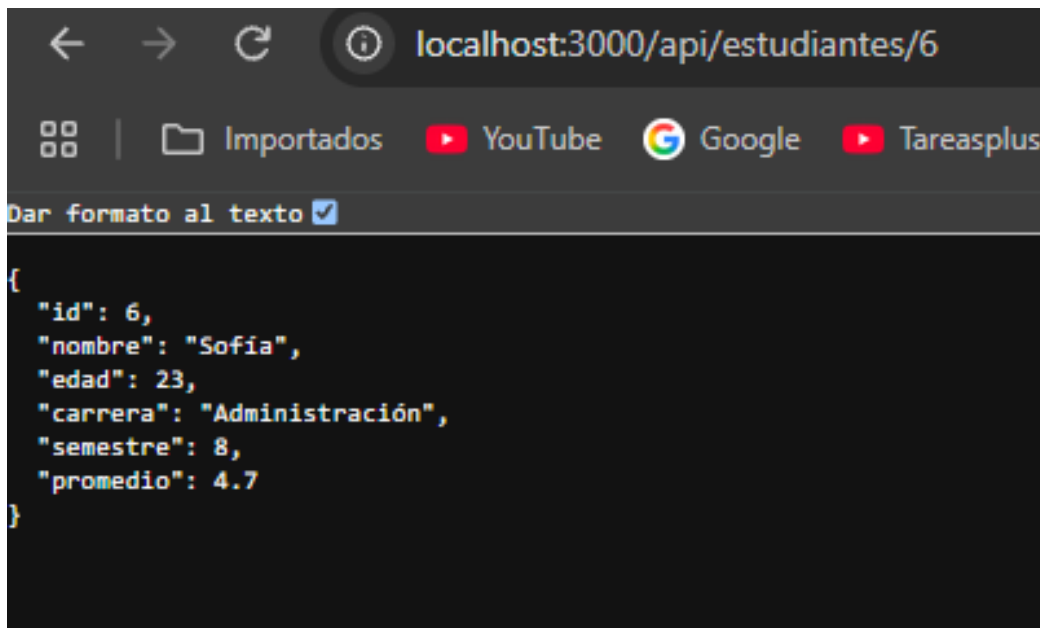
/api/estudiantes/4 — devuelve a Mariana



A screenshot of a web browser window. The address bar shows 'localhost:3000/api/estudiantes/4'. The browser's toolbar includes back, forward, and refresh buttons, along with search engines like YouTube, Google, and Tareasplus. A text formatting toolbar is visible above the response area. The response is a JSON object displayed in a dark-themed editor.

```
{
  "id": 4,
  "nombre": "Mariana",
  "edad": 22,
  "carrera": "Contaduría",
  "semestre": 6,
  "promedio": 4.5
}
```

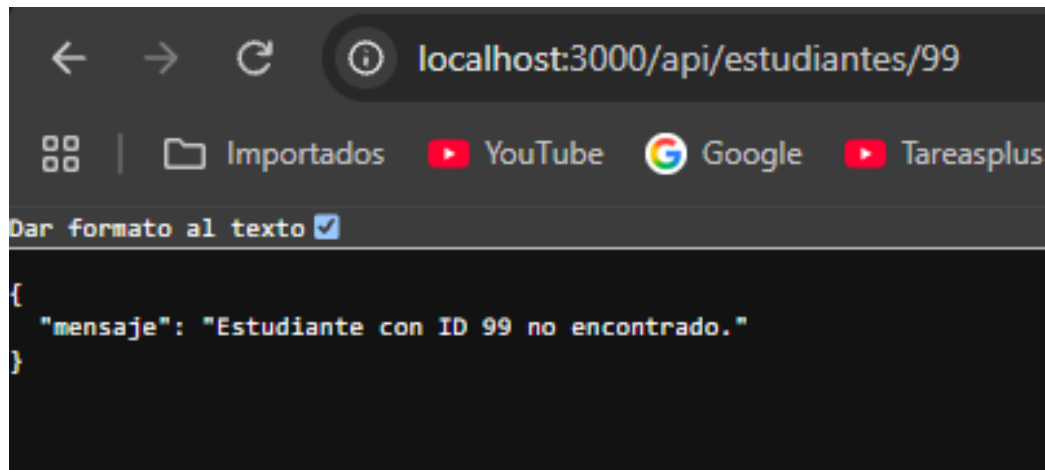
/api/estudiantes/6 — devuelve a Sofía



A screenshot of a web browser window. The address bar shows 'localhost:3000/api/estudiantes/6'. The browser's toolbar includes back, forward, and refresh buttons, along with search engines like YouTube, Google, and Tareasplus. A text formatting toolbar is visible above the response area. The response is a JSON object displayed in a dark-themed editor.

```
{
  "id": 6,
  "nombre": "Sofía",
  "edad": 23,
  "carrera": "Administración",
  "semestre": 8,
  "promedio": 4.7
}
```

/api/estudiantes/99 — ID inexistente, respuesta 404



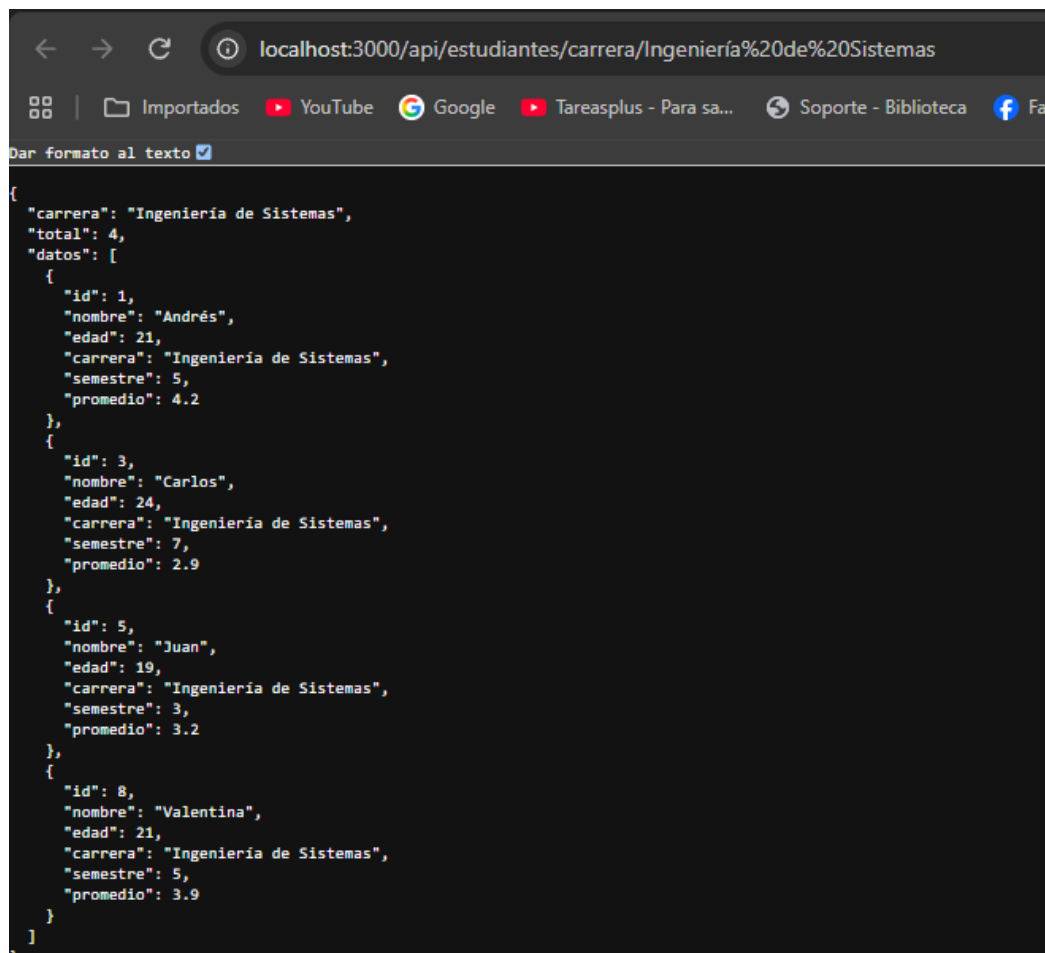
```
localhost:3000/api/estudiantes/99

{
  "mensaje": "Estudiante con ID 99 no encontrado."
}
```

Punto 3 — Buscar estudiantes por carrera

La búsqueda funciona con o sin tildes y sin distinguir mayúsculas. Si la carrera no existe se responde con un mensaje de error en lugar de una lista vacía.

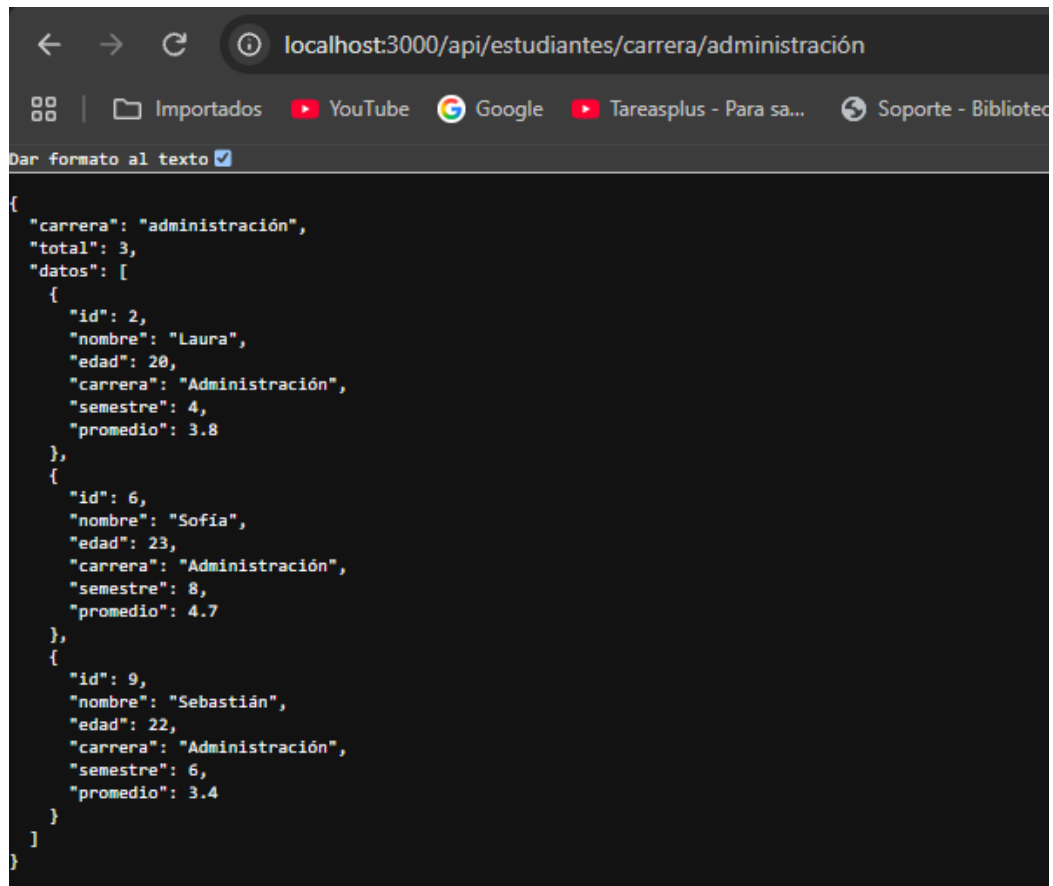
/api/estudiantes/carrera/Ingeniería de Sistemas — 4 estudiantes



```
localhost:3000/api/estudiantes/carrera/Ingeniería%20de%20Sistemas

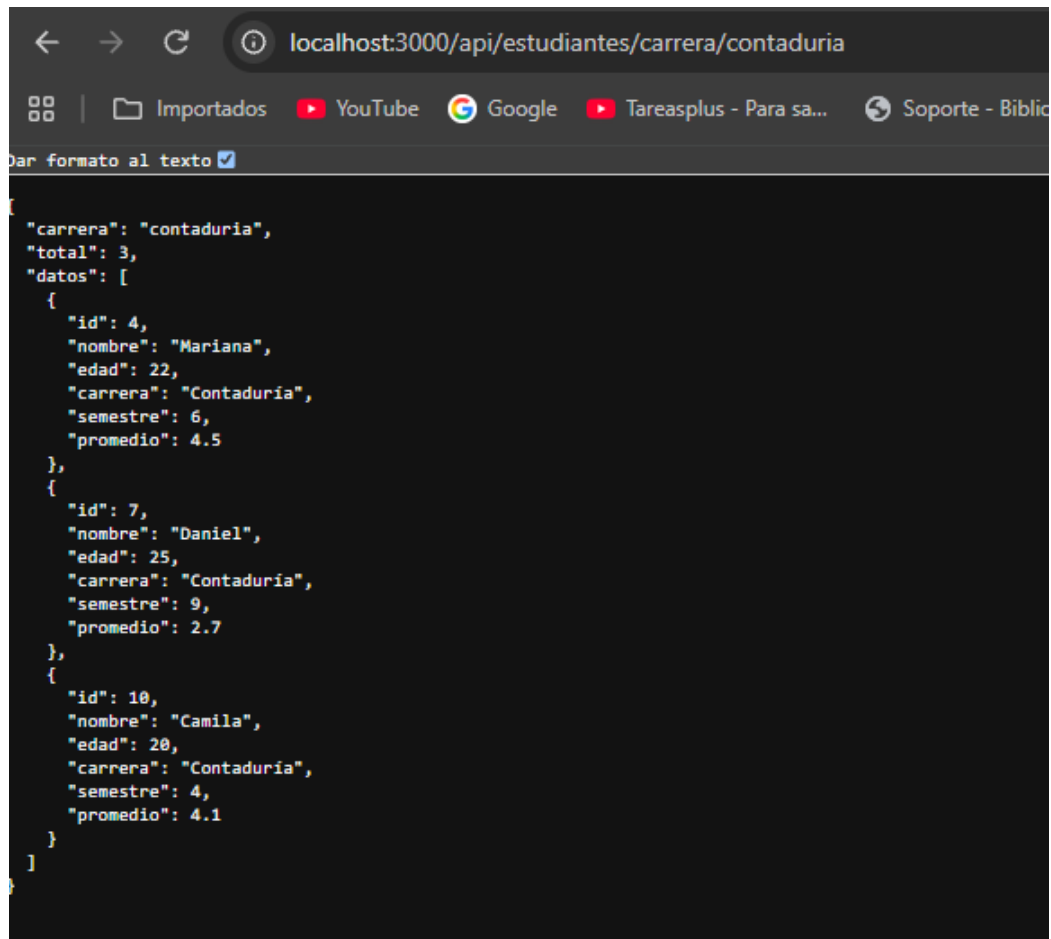
{
  "carrera": "Ingeniería de Sistemas",
  "total": 4,
  "datos": [
    {
      "id": 1,
      "nombre": "Andrés",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 5,
      "promedio": 4.2
    },
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    },
    {
      "id": 5,
      "nombre": "Juan",
      "edad": 19,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 3,
      "promedio": 3.2
    },
    {
      "id": 8,
      "nombre": "Valentina",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 5,
      "promedio": 3.9
    }
  ]
}
```

/api/estudiantes/carrera/administración — 3 estudiantes



```
{
  "carrera": "administración",
  "total": 3,
  "datos": [
    {
      "id": 2,
      "nombre": "Laura",
      "edad": 20,
      "carrera": "Administración",
      "semestre": 4,
      "promedio": 3.8
    },
    {
      "id": 6,
      "nombre": "Sofia",
      "edad": 23,
      "carrera": "Administración",
      "semestre": 8,
      "promedio": 4.7
    },
    {
      "id": 9,
      "nombre": "Sebastián",
      "edad": 22,
      "carrera": "Administración",
      "semestre": 6,
      "promedio": 3.4
    }
  ]
}
```

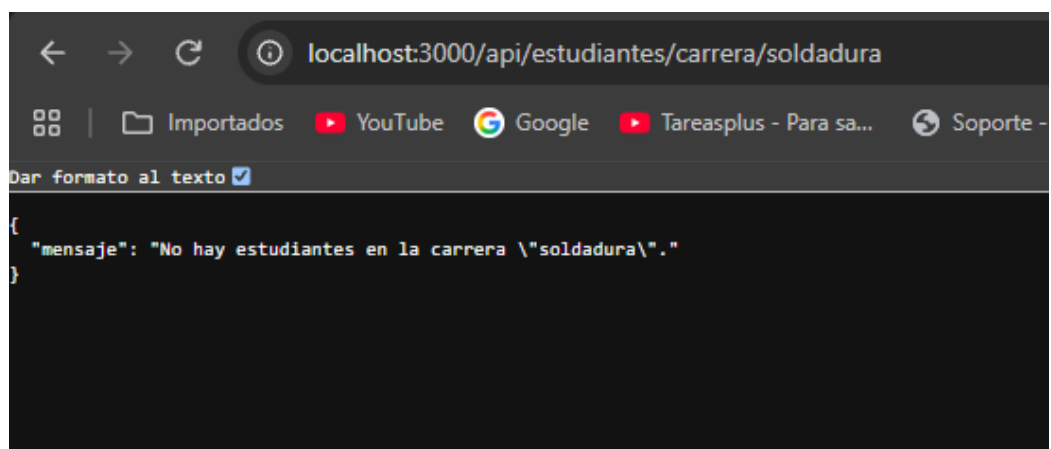
/api/estudiantes/carrera/contaduria — escrito sin tilde, encuentra los 3



A screenshot of a web browser window. The address bar shows the URL `localhost:3000/api/estudiantes/carrera/contaduria`. The browser's developer tools are open, displaying a JSON response from the API. The response is a valid JSON object with a 'carrera' field, a 'total' field, and a 'datos' array containing three student records. The browser's address bar also shows several tabs: 'Importados', 'YouTube', 'Google', 'Tareasplus - Para sa...', and 'Soporte - Biblio...'. The browser's interface is in Spanish.

```
{
  "carrera": "contaduria",
  "total": 3,
  "datos": [
    {
      "id": 4,
      "nombre": "Mariana",
      "edad": 22,
      "carrera": "Contaduria",
      "semestre": 6,
      "promedio": 4.5
    },
    {
      "id": 7,
      "nombre": "Daniel",
      "edad": 25,
      "carrera": "Contaduria",
      "semestre": 9,
      "promedio": 2.7
    },
    {
      "id": 10,
      "nombre": "Camila",
      "edad": 20,
      "carrera": "Contaduria",
      "semestre": 4,
      "promedio": 4.1
    }
  ]
}
```

`/api/estudiantes/carrera/soldadura` — carrera inexistente, respuesta 404



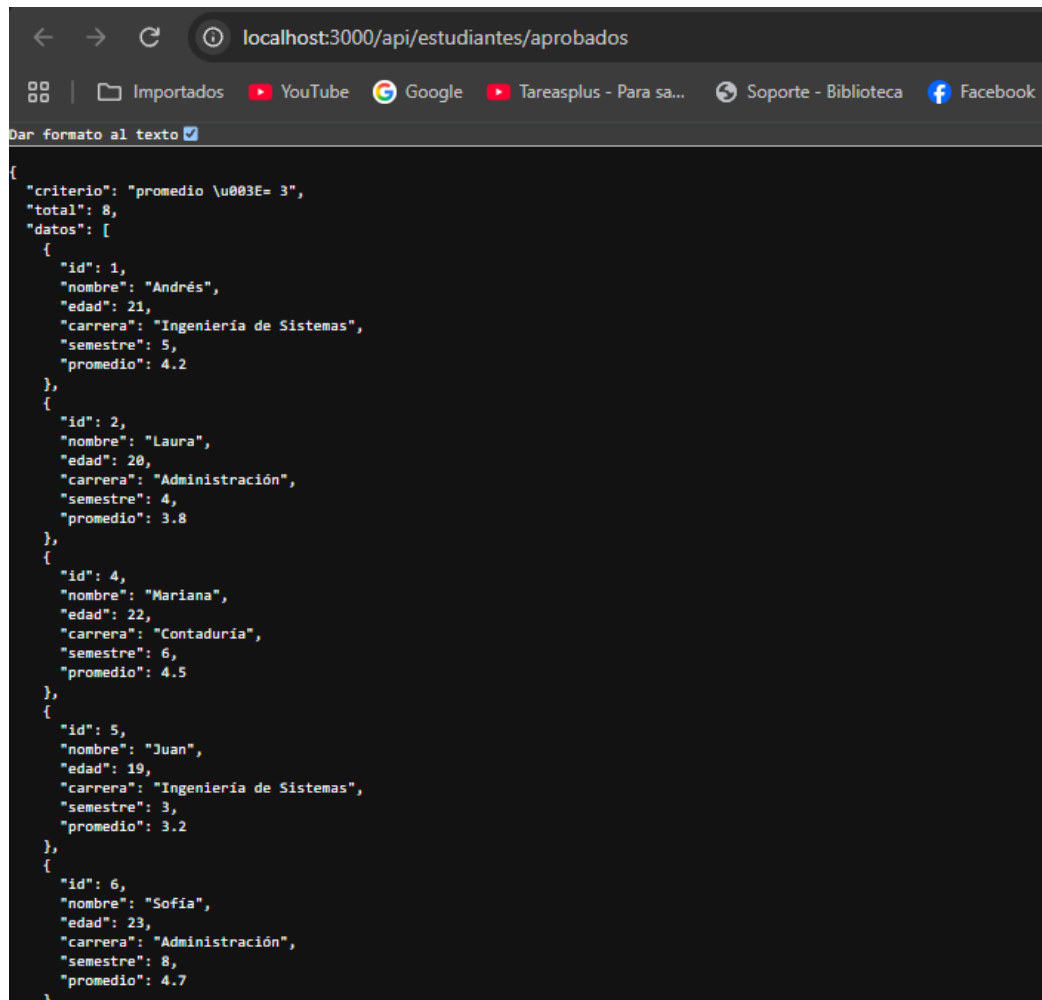
A screenshot of a web browser window. The address bar shows the URL `localhost:3000/api/estudiantes/carrera/soldadura`. The browser's developer tools are open, displaying a JSON response from the API. The response is a valid JSON object with a 'mensaje' field containing the text "No hay estudiantes en la carrera \"soldadura\".". The browser's address bar also shows several tabs: 'Importados', 'YouTube', 'Google', 'Tareasplus - Para sa...', and 'Soporte - ...'. The browser's interface is in Spanish.

```
{
  "mensaje": "No hay estudiantes en la carrera \"soldadura\"."
}
```

Punto 4 — Estudiantes aprobados

La respuesta incluye el criterio aplicado, el total y el listado.

`/api/estudiantes/aprobados` — 8 estudiantes con promedio igual o superior a 3.0

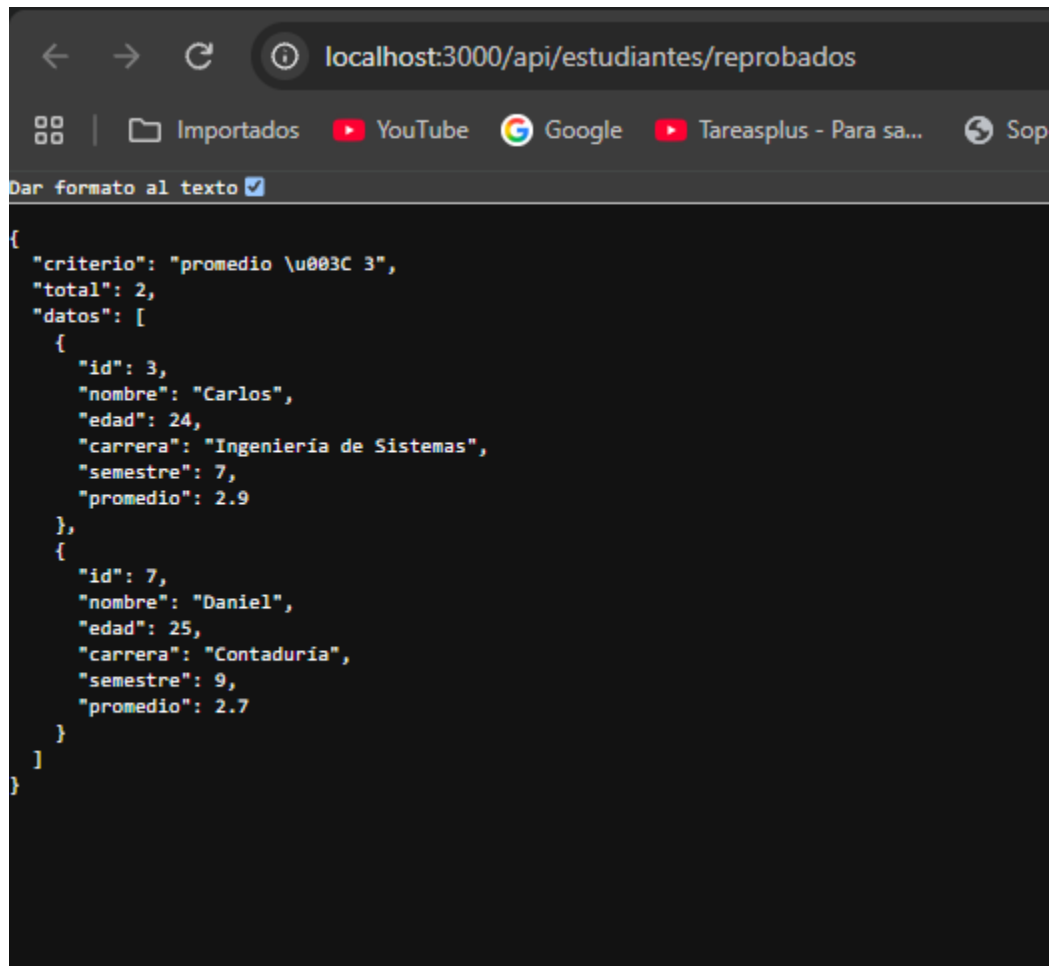


```
{
  "criterio": "promedio \u003E= 3",
  "total": 8,
  "datos": [
    {
      "id": 1,
      "nombre": "Andr\u00e9s",
      "edad": 21,
      "carrera": "Ingenier\u00eda de Sistemas",
      "semestre": 5,
      "promedio": 4.2
    },
    {
      "id": 2,
      "nombre": "Laura",
      "edad": 20,
      "carrera": "Administraci\u00f3n",
      "semestre": 4,
      "promedio": 3.8
    },
    {
      "id": 4,
      "nombre": "Mariana",
      "edad": 22,
      "carrera": "Contadur\u00eda",
      "semestre": 6,
      "promedio": 4.5
    },
    {
      "id": 5,
      "nombre": "Juan",
      "edad": 19,
      "carrera": "Ingenier\u00eda de Sistemas",
      "semestre": 3,
      "promedio": 3.2
    },
    {
      "id": 6,
      "nombre": "Sof\u00eda",
      "edad": 23,
      "carrera": "Administraci\u00f3n",
      "semestre": 8,
      "promedio": 4.7
    }
  ]
}
```

Punto 5 — Estudiantes reprobados

Misma estructura de respuesta con el criterio invertido.

/api/estudiantes/reprobados — 2 estudiantes por debajo de 3.0

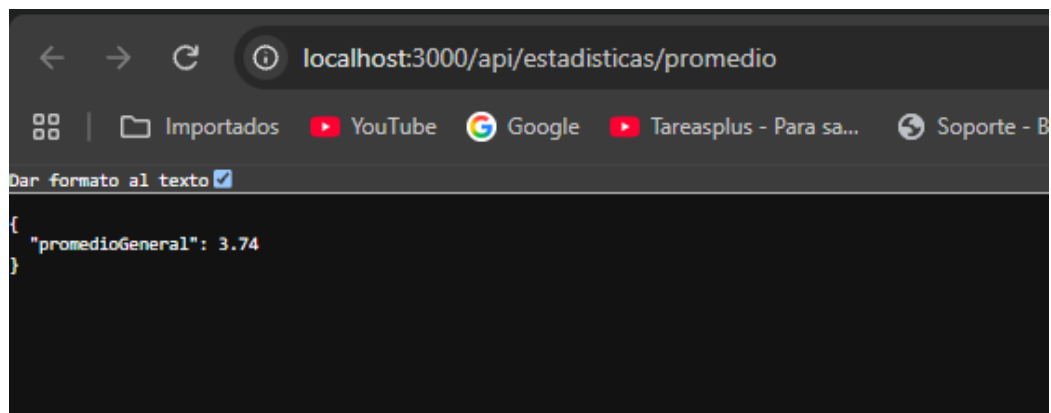


```
{
  "criterio": "promedio \u003C 3",
  "total": 2,
  "datos": [
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingenier\u00eda de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    },
    {
      "id": 7,
      "nombre": "Daniel",
      "edad": 25,
      "carrera": "Contadur\u00eda",
      "semestre": 9,
      "promedio": 2.7
    }
  ]
}
```

Punto 6 — Promedio general

Devuelve un \u00fanico valor num\u00e9rico ya redondeado a dos decimales.

/api/estadisticas/promedio — 3.74

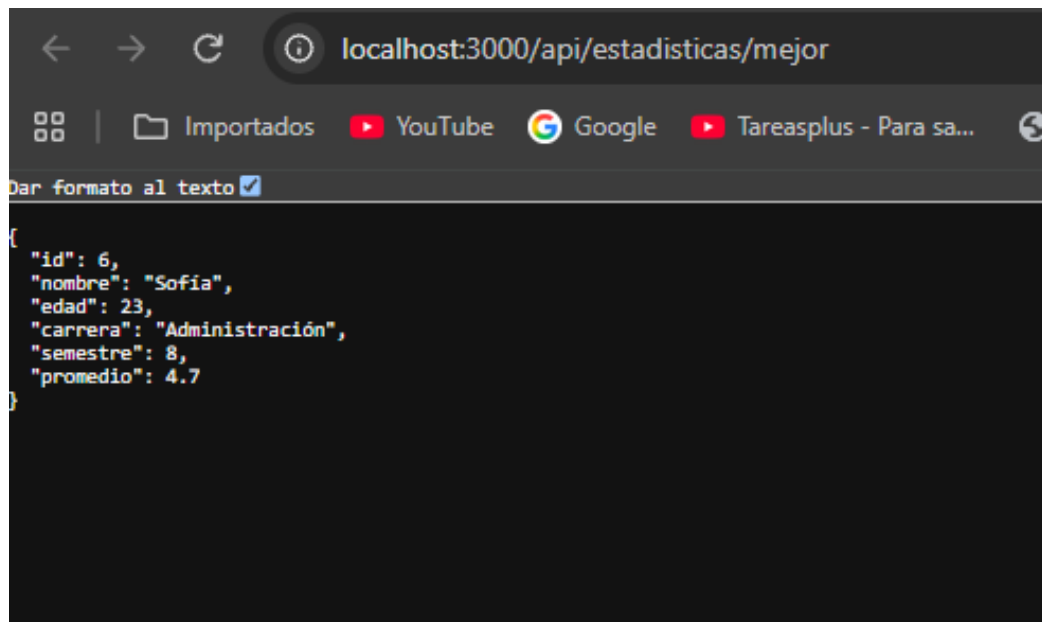


```
{
  "promedioGeneral": 3.74
}
```

Puntos 7 y 8 — Mejor y peor estudiante

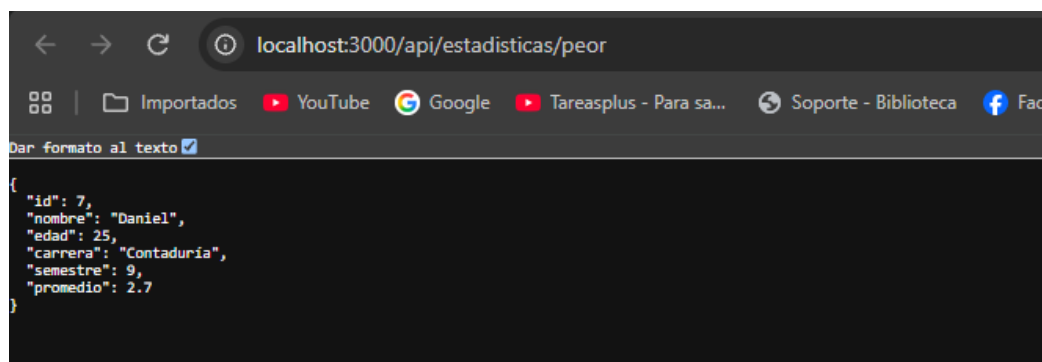
Cada endpoint devuelve el objeto completo del estudiante correspondiente.

/api/estadisticas/mejor — Sofía con 4.7



```
{
  "id": 6,
  "nombre": "Sofia",
  "edad": 23,
  "carrera": "Administración",
  "semestre": 8,
  "promedio": 4.7
}
```

/api/estadisticas/peor — Daniel con 2.7

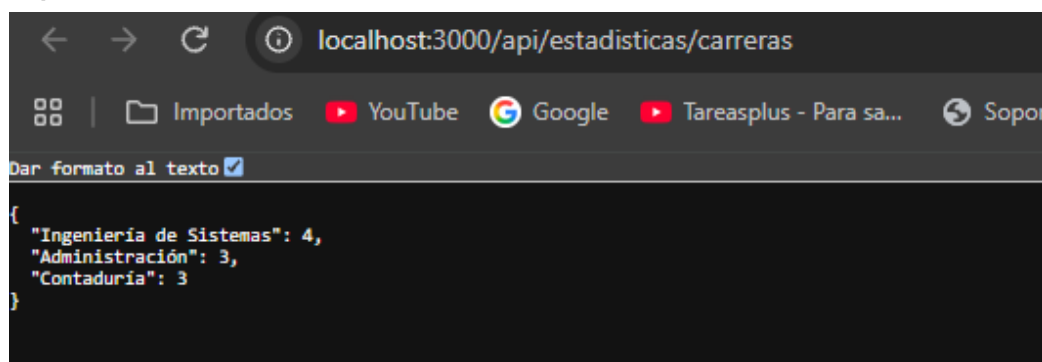


```
{
  "id": 7,
  "nombre": "Daniel",
  "edad": 25,
  "carrera": "Contaduría",
  "semestre": 9,
  "promedio": 2.7
}
```

Punto 9 — Cantidad de estudiantes por carrera

El objeto devuelto usa los nombres de las carreras como llaves.

/api/estadisticas/carreras — conteo por carrera

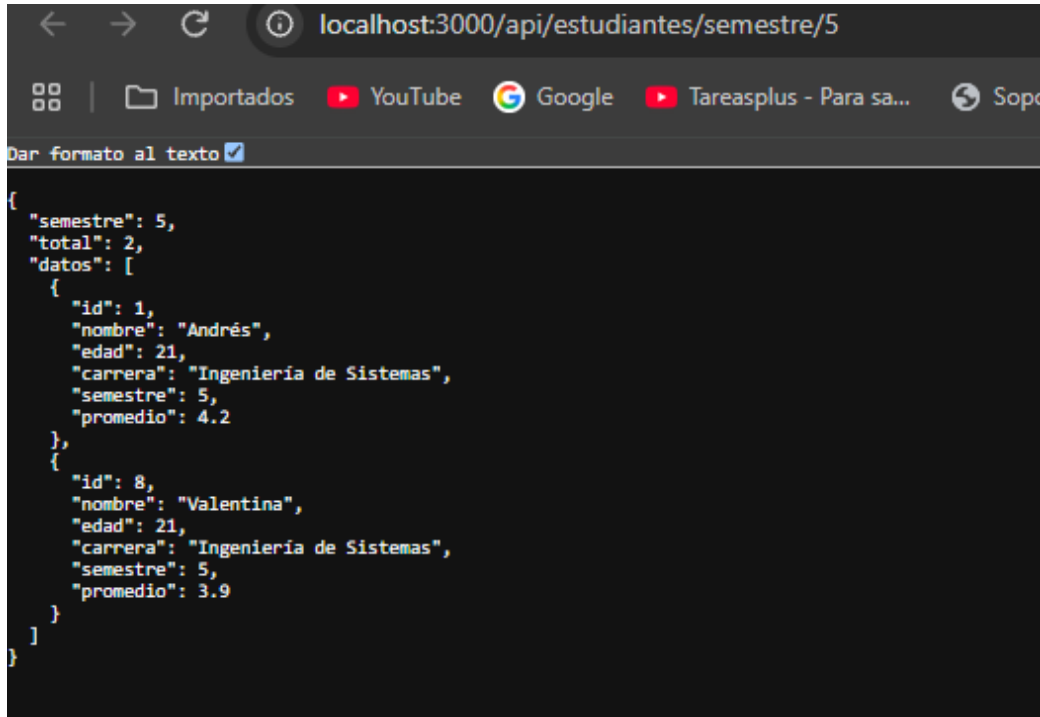


```
{
  "Ingenieria de Sistemas": 4,
  "Administración": 3,
  "Contaduría": 3
}
```

Punto 10 — Buscar estudiantes por semestre

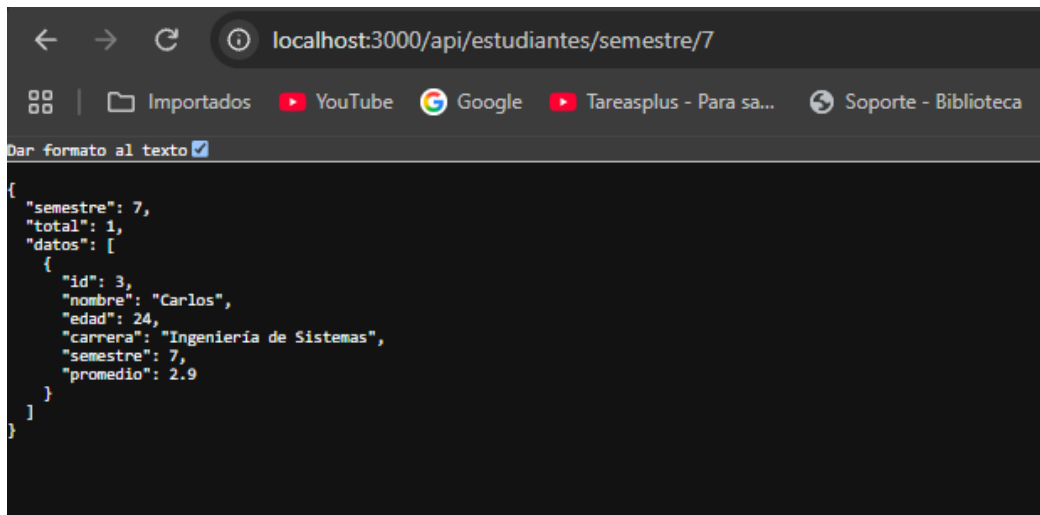
Cuando ningún estudiante cursa el semestre consultado, la respuesta incluye un total en cero y un arreglo vacío.

/api/estudiantes/semestre/5 — 2 estudiantes



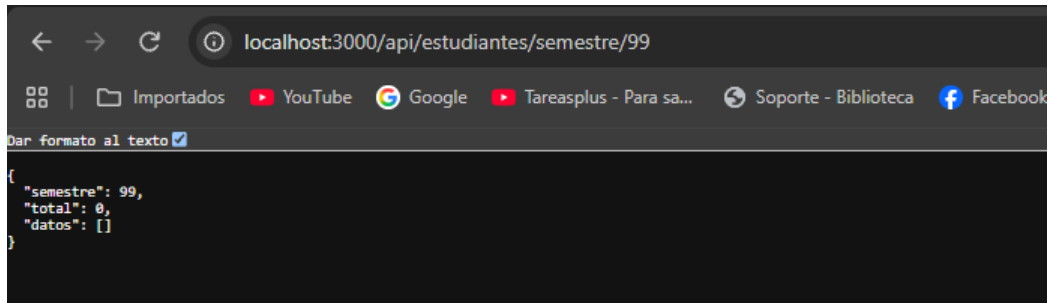
```
{
  "semestre": 5,
  "total": 2,
  "datos": [
    {
      "id": 1,
      "nombre": "Andrés",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 5,
      "promedio": 4.2
    },
    {
      "id": 8,
      "nombre": "Valentina",
      "edad": 21,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 5,
      "promedio": 3.9
    }
  ]
}
```

/api/estudiantes/semestre/7 — 1 estudiante



```
{
  "semestre": 7,
  "total": 1,
  "datos": [
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    }
  ]
}
```

/api/estudiantes/semestre/99 — semestre inexistente, arreglo vacío

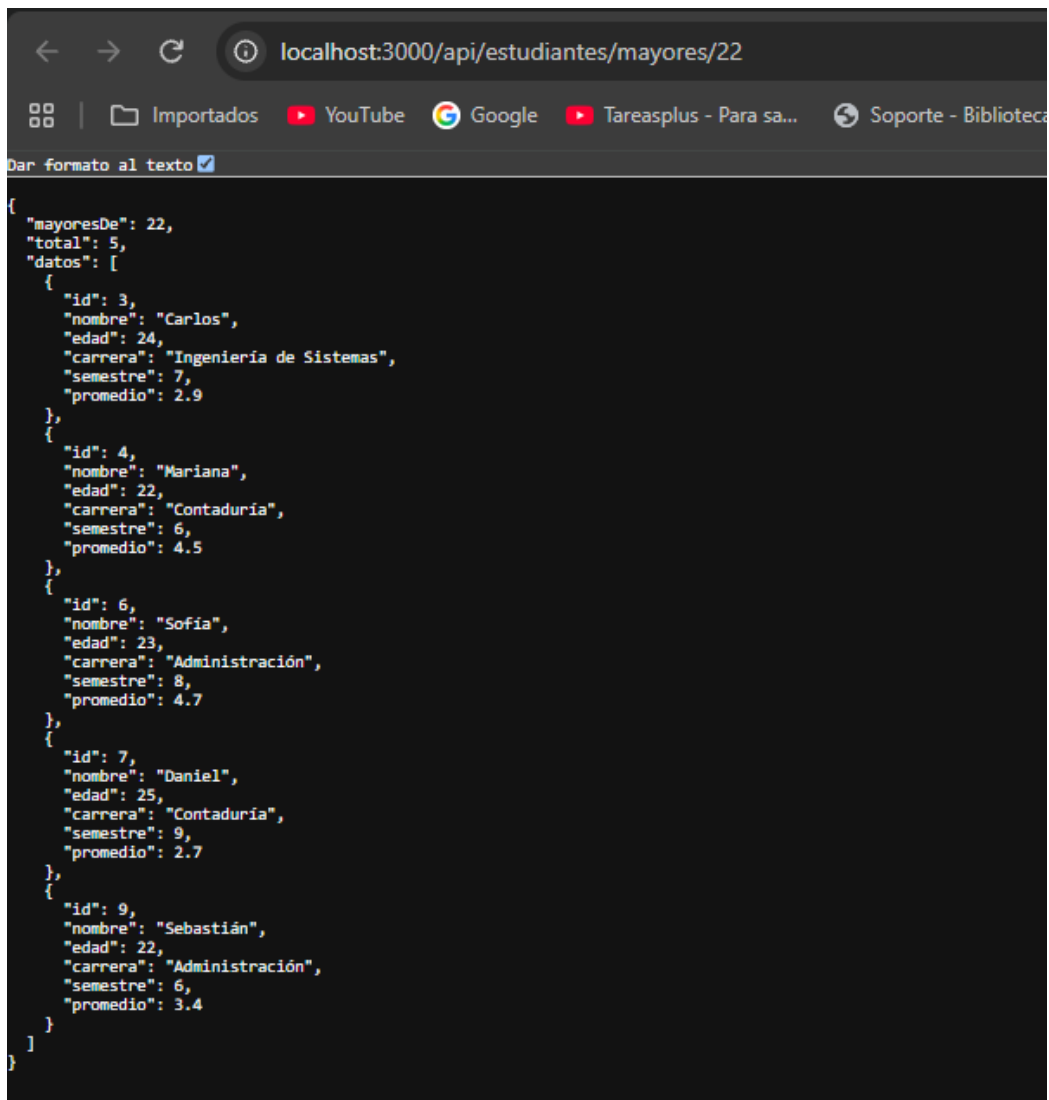


```
{
  "semestre": 99,
  "total": 0,
  "datos": []
}
```

Punto 11 — Estudiantes mayores de cierta edad

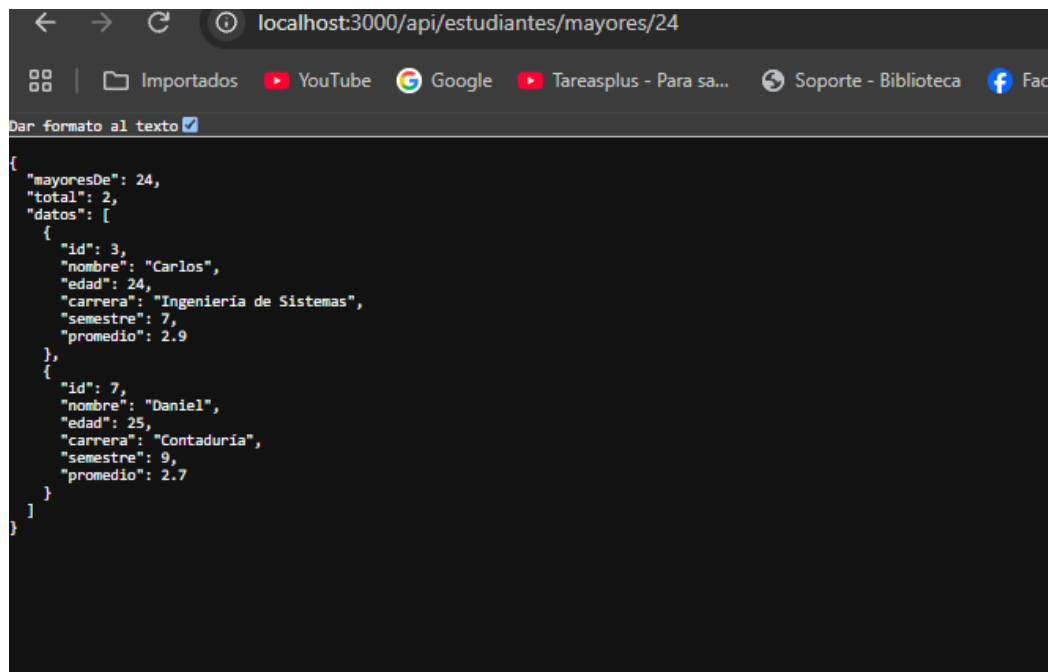
El límite de edad viaja como parámetro en la URL, igual que el identificador del punto 2.

/api/estudiantes/mayores/22 — mayores de 22 años



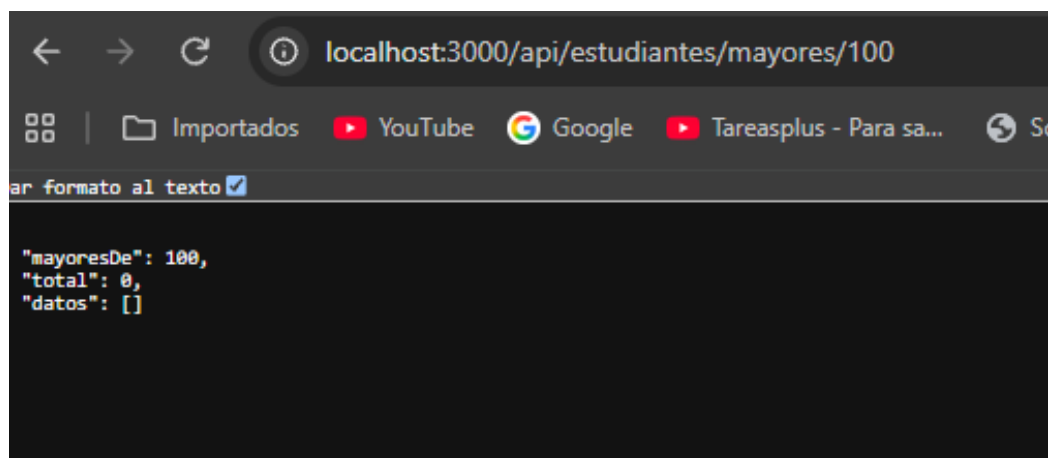
```
{
  "mayoresDe": 22,
  "total": 5,
  "datos": [
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    },
    {
      "id": 4,
      "nombre": "Mariana",
      "edad": 22,
      "carrera": "Contaduría",
      "semestre": 6,
      "promedio": 4.5
    },
    {
      "id": 6,
      "nombre": "Sofía",
      "edad": 23,
      "carrera": "Administración",
      "semestre": 8,
      "promedio": 4.7
    },
    {
      "id": 7,
      "nombre": "Daniel",
      "edad": 25,
      "carrera": "Contaduría",
      "semestre": 9,
      "promedio": 2.7
    },
    {
      "id": 9,
      "nombre": "Sebastián",
      "edad": 22,
      "carrera": "Administración",
      "semestre": 6,
      "promedio": 3.4
    }
  ]
}
```

/api/estudiantes/mayores/24 — mayores de 24 años



```
{
  "mayoresDe": 24,
  "total": 2,
  "datos": [
    {
      "id": 3,
      "nombre": "Carlos",
      "edad": 24,
      "carrera": "Ingeniería de Sistemas",
      "semestre": 7,
      "promedio": 2.9
    },
    {
      "id": 7,
      "nombre": "Daniel",
      "edad": 25,
      "carrera": "Contaduría",
      "semestre": 9,
      "promedio": 2.7
    }
  ]
}
```

/api/estudiantes/mayores/100 — ningún estudiante supera esa edad

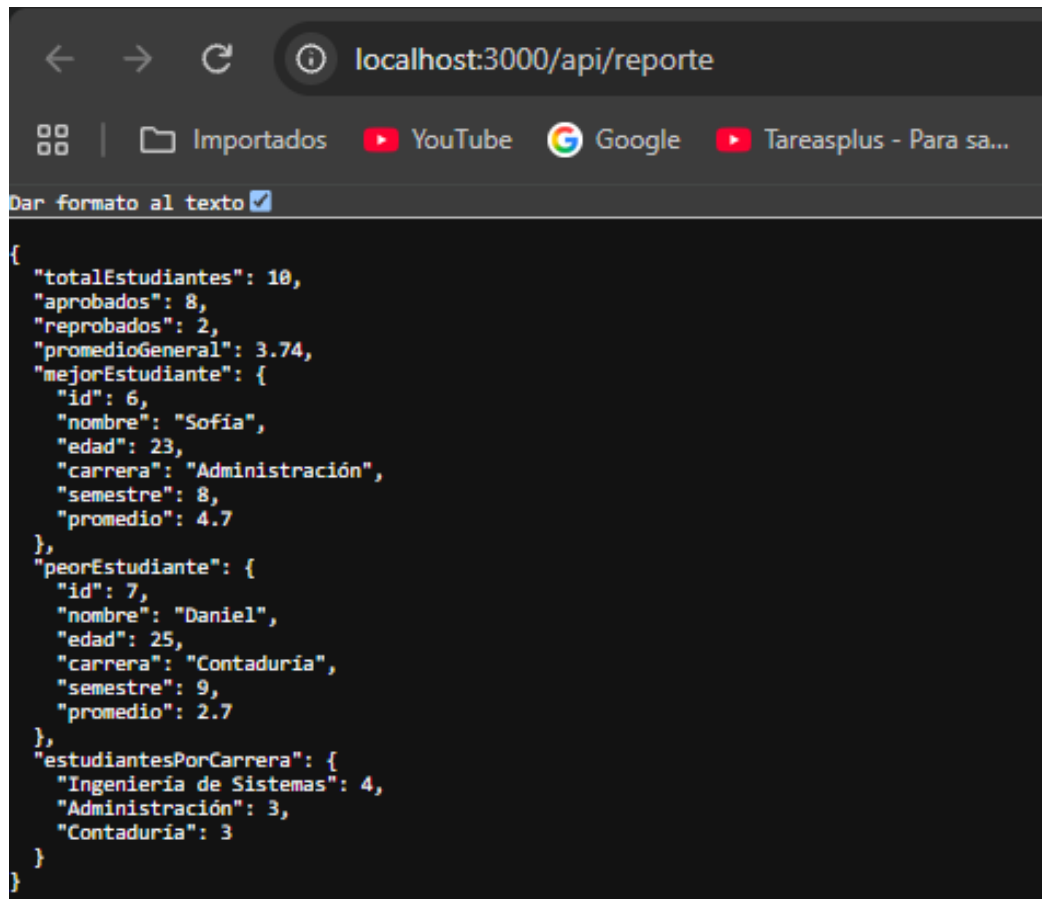


```
{
  "mayoresDe": 100,
  "total": 0,
  "datos": []
}
```

Punto 12 — Reporte general

El mismo reporte que se imprime con formato de texto en la consola se entrega aquí como un objeto JSON, generado por la misma función.

/api/reporte — reporte académico completo



```
{
  "totalEstudiantes": 10,
  "aprobados": 8,
  "reprobados": 2,
  "promedioGeneral": 3.74,
  "mejorEstudiante": {
    "id": 6,
    "nombre": "Sofia",
    "edad": 23,
    "carrera": "Administración",
    "semestre": 8,
    "promedio": 4.7
  },
  "peorEstudiante": {
    "id": 7,
    "nombre": "Daniel",
    "edad": 25,
    "carrera": "Contaduría",
    "semestre": 9,
    "promedio": 2.7
  },
  "estudiantesPorCarrera": {
    "Ingeniería de Sistemas": 4,
    "Administración": 3,
    "Contaduría": 3
  }
}
```

Reto adicional — Ranking por promedio

Devuelve los estudiantes ordenados de mayor a menor promedio, con su posición en el ranking.

/api/ranking — los 10 estudiantes ordenados

← → ↻ ⓘ localhost:3000/api/ranking

🗑 Importados 📺 YouTube 🌐 Google 📺 Tareasplus - Para sa... 🔄 Soporte

Dar formato al texto ☒

```
[
  {
    "posicion": 1,
    "nombre": "Sofía",
    "promedio": 4.7
  },
  {
    "posicion": 2,
    "nombre": "Mariana",
    "promedio": 4.5
  },
  {
    "posicion": 3,
    "nombre": "Andrés",
    "promedio": 4.2
  },
  {
    "posicion": 4,
    "nombre": "Camila",
    "promedio": 4.1
  },
  {
    "posicion": 5,
    "nombre": "Valentina",
    "promedio": 3.9
  },
  {
    "posicion": 6,
    "nombre": "Laura",
    "promedio": 3.8
  },
  {
    "posicion": 7,
    "nombre": "Sebastián",
    "promedio": 3.4
  },
  {
    "posicion": 8,
    "nombre": "Juan",
    "promedio": 3.2
  },
  {
    "posicion": 9,
    "nombre": "Carlos",
    "promedio": 2.9
  },
  {
    "posicion": 10,
    "nombre": "Daniel",
    "promedio": 2.7
  }
]
```

4. Conclusión

Los doce requerimientos del enunciado y el reto adicional quedaron implementados y verificados, tanto de forma individual como en la ejecución completa del programa.

Las mismas funciones alimentan las dos salidas del proyecto. Las capturas de la sección 2 y las de la sección 3 muestran resultados idénticos obtenidos desde la consola y desde la API, sin que exista lógica duplicada entre ambas.

Esto fue posible porque las funciones retornan datos en lugar de imprimirlos. El caso más claro es la búsqueda por identificador: cuando el estudiante no existe, la función devuelve null, la consola lo traduce en un mensaje de no encontrado y Express lo traduce en un código HTTP 404.

Se probaron también los casos sin resultados (carreras, semestres e identificadores inexistentes) para verificar que el programa responde de forma controlada y no con un error de ejecución.